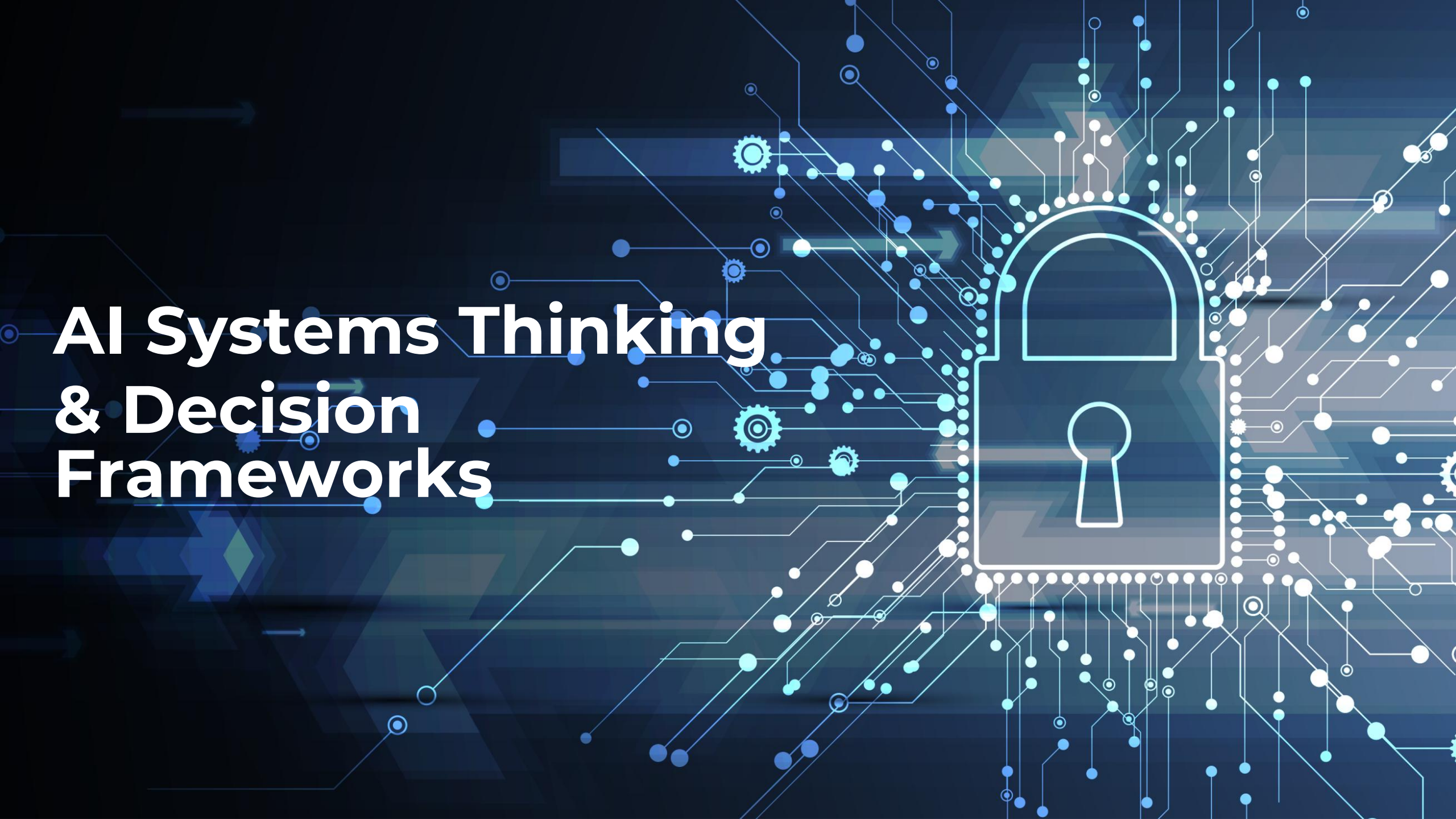
IITM PRAVARTAK
CATALYSING INNOVATION

Advanced Certificate Programme in Agentic AI & RAG Engineering

30-WEEK PROGRAM

AI Systems Thinking & Decision Frameworks



DAY 1

What we're building, and your first call

Concepts → mental model + four patterns. Hands-on → Hello LLM live-along.

Welcome to the Program

A 30-week build, one shared capstone

30
weeks, fully live

1
shared capstone

7
design reviews

This is a design course as much as a coding course.

- Every week is live, hands-on, and gently paced — no prior LLM experience assumed.
- Each session adds one real piece to a single capstone you carry the whole way.
- By Week 30 you ship a working system and a portfolio that shows how you got there.

Today — Day 1

Three hours together — what we're building, and your first call

0:00	Welcome, cohort norms & 30-second intros
0:15	What “agentic AI” actually means
0:50	Break
1:05	Choosing the right tool — four patterns
1:50	Hello LLM — your first API call together
2:35	Discussion + Day 1 wrap

How We Work Together

Three norms that make a live cohort succeed

1

“I don’t understand” is a strength

Honest questions move the whole cohort forward — they are never a weakness.

2

Critique ideas, not people

We are all here to build. Respectful, specific feedback is how the work gets better.

3

Consistency over brilliance

Showing up every week beats being the smartest person in Week 1.

30-Second Intros

Around the room — keep it quick

Your name

- Where you're joining from
- Time zone (for study groups)

Your current role

- What you do day to day
- How much you code now

One thing to build

- A problem you'd love to solve
- Or simply what you want to learn

What “Agentic AI” Actually Means

A plain-language framing — and the system we’ll build together.

The Capability Spectrum

From simple replies to systems that decide

1

Chatbot

Scripted, rule-based replies

e.g. FAQ bot

2

Q&A app

Answers from a fixed prompt

e.g. ChatGPT

3

Workflow automation

Fixed multi-step pipeline

e.g. doc summariser

4

Agentic system

Decides its own next step

e.g. coding agent

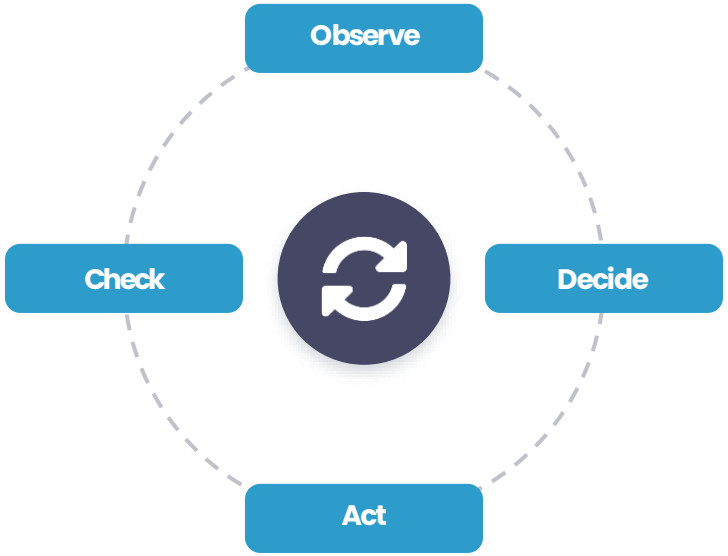
Each step adds autonomy. Most production value sits in the middle today — we build toward the right edge.

A Working Definition

Hold onto this one sentence

An agent is a loop that uses an LLM to decide what to do next.

Everything in the next 30 weeks is about making that loop reliable, grounded, observable, and affordable.



Our Capstone — Knowledge Assistant

One system, all 30 weeks

A Q&A system over a body of internal-style documents — ask in plain language, get a grounded answer with sources.

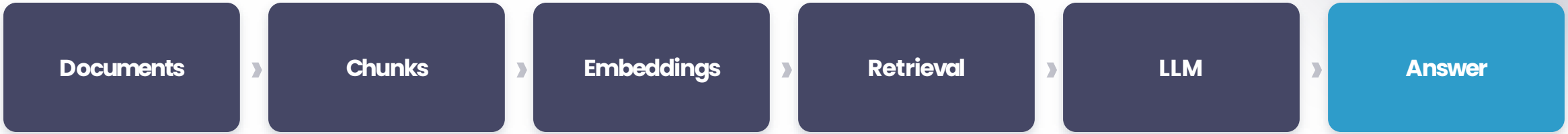
- Every learner builds the same architecture.
- You choose your own document corpus.
- It grows piece by piece toward a production system.

EXAMPLE QUESTIONS

- “What’s our leave-carryover policy?”
- “Which regulation covers data retention?”
- “Summarise the onboarding steps.”

The Honest Architecture Sketch

Where every piece will live



We have NOT built any of this yet. We add one piece at a time over 30 weeks. Today we build only the very first arrow — a single call to the LLM.

An Honest 30-Week Roadmap

Seven phases, building to a deployed system

Phase	Focus	Weeks
P1	Foundations & system design	W1–5
P2	Retrieval (RAG) from scratch	W6–12
P3	Agents	W13–18
P4	Multi-agent	W19–22
P5	MCP & integration	W23–24
P6	Observability & cost	W25–27
P7	Production, compliance & demo	W28–30

By W30: a deployed system with all 8 KPIs reported — and a portfolio that proves the journey.

Shared Architecture, Your Corpus

Same engineering — your choice of documents

HR policies

Product manuals

Public regulatory PDFs

Open-source wikis

Rule: 5–20 documents, and NO real PII or confidential data. If you'd hesitate to let a peer review it, swap it.

Break

15 minutes — back at the top of the hour.

Choosing the Right Tool

Four architecture patterns — a reference sheet, not a debate.

Four Patterns to Engineering AI(LLM Powered) Solutions

Each fits a different shape of problem

RAG

Retrieval-Augmented Generation

Fine-tuning

Teach a stable style or skill

Long-context

Stuff it all in the prompt

Agent frameworks

Tools + multi-step planning

Next: one slide each — when to use it, when not, and the rough cost.

RAG — Retrieval-Augmented Generation

PATTERN 1 OF 4

✓ USE WHEN

- Knowledge changes often
- You need answers grounded in your own documents
- You must be able to cite sources

✕ SKIP WHEN

- The knowledge is tiny and static
- You need a new behaviour or skill, not facts

COST / MAINTENANCE

Low–moderate. Re-index when docs change; no model training.

EXAMPLE

Our capstone — Q&A over internal docs.

Fine-Tuning

PATTERN 2 OF 4

✓ USE WHEN

- You need a stable style, tone, or format
- A narrow, repeated skill
- Latency or cost of long prompts hurts

✕ SKIP WHEN

- Facts change frequently
- You have little labelled data
- You just need knowledge → use RAG

COST / MAINTENANCE

High up-front. Data prep + training + re-training on drift.

EXAMPLE

A model that always replies in your brand voice.

Long-Context Prompting

PATTERN 3 OF 4

✓ USE WHEN

- Content is small and self-contained
- It fits comfortably in the prompt
- One-off or low-volume tasks

✕ SKIP WHEN

- Knowledge is large or grows
- High request volume (cost per call adds up)

COST / MAINTENANCE

Pay per token, every call. Simple to build, scales poorly.

EXAMPLE

Summarise one pasted contract.

Agent Frameworks

PATTERN 4 OF 4

✓ USE WHEN

- The task needs tools (search, code, APIs)
- Multi-step planning is required
- The next step depends on prior results

✕ SKIP WHEN

- A single prompt would do
- Determinism & low latency matter most

COST / MAINTENANCE

Highest. More LLM calls, harder to debug.

EXAMPLE

A coding agent that runs and fixes tests.

Your Decision Sheet

You'll consult this all program

Pattern	Reach for it when...	Cost
RAG	Knowledge is dynamic; answers must be grounded	Low–Mid
Fine-tuning	You need a stable style or a narrow skill	High
Long-context	Context is small and self-contained	Per-call
Agent frameworks	Task needs tools or multi-step planning	Highest

There is no "best" pattern — only the right one for the constraints in front of you.

Hello LLM

Your first API call — everyone leaves today with a working script.

Setup — Two Minutes

Follow along on Vocareum

Open a terminal	We're on Vocareum — Python 3.11 is ready.
Install packages	`pip install openai python-dotenv`
API key is configured	Vocareum has OPENAI_API_KEY set in your environment.
You'll build hello_llm.py	Step-by-step walkthrough in this week's lab guide.

On your own machine, put the key in a .env file — never commit it. (Security depth lands in W8 & W28.)

In the Lab — Build hello_llm.py

Your first real LLM call — written by you, step by step

~20 lines that:

1. Load OPENAI_API_KEY from .env
2. Create an OpenAI client
3. Define an `ask(question)` function
4. Run it from the command line

Lab Step 2 walks you through it:

- 2a — Install packages.
- 2b — Create the file (imports + .env loading).
- 2c — Author the ask() function.
- 2d — Add __main__ and run it.
- 2e — Run three times, save outputs.

Each sub-step explains the goal, the code, the run, and what the output means.

What Just Happened?

One request, one response

~$0.0001

cost per call

~1–3 s

round-trip latency

1 → 1

request to response

That's the whole foundation. In W2 we make this asynchronous, retried, and logged — real engineering on top of this one call.

Day 1 — wrap

What we covered, what's next tomorrow

Today, you...

- Built a mental model — what an agent is, where things sit on the spectrum.
- Learned the four architecture patterns — RAG, fine-tuning, long-context, agent frameworks.
- Saw what your first LLM call looks like — you'll build hello_llm.py in tonight's lab.

Tomorrow we...

- Measure success — the eight production KPIs.
- Decide when NOT to use AI.
- Write your first ADR using the Solution Framing Canvas.
- Walk through this week's lab.

DAY 2

Measuring, deciding, documenting

Concepts → 8 KPIs + when NOT to use AI. Hands-on → fill the Solution Framing Canvas live.

Welcome back

A quick check-in before we dive in

Did your hello_llm.py
run cleanly?

Did you save your three
runs from yesterday?

Anything still stuck
on Vocareum / .env?

Today we add the engineering thinking.

- Yesterday was about WHAT we're building and WHICH tool fits.
- Today is about HOW WELL it's working, WHEN NOT to use AI, and HOW to document the decision.
- By the end, you've drafted your first Architecture Decision Record.

Today — Day 2

Three hours together — measure, decide, document

0:00	Welcome back + Day 1 recap
0:15	Eight production KPIs
0:45	When NOT to use AI
1:00	Break
1:15	The Living ADR
1:35	Solution Framing Canvas + worked example (live)
2:15	This week's lab — walkthrough
2:45	Key takeaways + Q&A

Measuring Success

Eight KPIs — and the discipline of knowing when NOT to use AI.

The Eight Production KPIs

Shared language now — real numbers from W6

1
Task success
Did it do the job?

2
Grounded response
Backed by sources?

3
Hallucination rate
How often it invents

4
Retrieval hit rate
Right docs found?

5
Tool success rate
Tools called correctly

6
Cost per query
What each answer costs

7
Latency p50 / p95
How fast it responds

8
Escalation rate
How often it hands off

No live measurement today — these become the scoreboard once the system has parts to measure (W6+).

When NOT to Use AI

Three everyday examples

Look up an order status

› **Use a database**

Determinism

Compute a tax bill

› **Use a calculator**

Regulation + exactness

Yesterday's stock close

› **Use an API**

Accuracy + cost

If cost, latency, regulation, or determinism disqualifies AI — choose the boring tool. It's the right call.

A Recurring Thread

This question never goes away

“When NOT to use AI” comes back in every phase.

- Today: AI vs. a database, a calculator, or an API.
- Later: when NOT to add memory (W15), go multi-agent (W19), or reach for MCP (W23).

Good engineering is knowing when the simplest tool wins.

Break

15 minutes — ADR and Canvas next..

Decide & Document

The Living ADR and the Solution Framing Canvas — your first design artifact.

The Living ADR

Architecture Decision Record

Lightweight

A one-pager in Markdown — not a thesis.

Living

It changes as you learn: v1 today → vN by W27.

Defensible

You explain your choices at each design review.

Today you write ADR v1. It becomes the spine of your portfolio.

Solution Framing Canvas

The shape of every agentic design — seven fields

1 Inputs
What goes in?

2 Tools
What can it call?

3 Memory
What does it remember?

4 Outputs
What comes out?

5 Autonomy level
How much can it act?

6 Decision boundaries
What's off-limits?

7 Success metrics
How do we judge it?

Fill all seven for your capstone — that becomes ADR v1.

Worked Example — Capstone Canvas

How the seven fields look, filled in

Field	Capstone — Knowledge Assistant (v1)
Inputs	Document corpus + user questions
Tools	Retrieval + a single LLM call
Memory	None in v1 — we add it later
Outputs	A grounded answer with citations
Autonomy level	Suggest only — no destructive actions
Decision boundaries	Answer only from the provided documents
Success metrics	Grounded response · hallucination rate · task success

v1 is deliberately small. "Suggest only" + "only from documents" is how scope stays safe while we learn.

This Week's Lab

≈ 2 hours · three steps · all on Vocareum

1

Set up your capstone repo

GitHub repo with README, src/, docs/ and a Python .gitignore. Name your capstone.

~30 min

2

Commit your first LLM script

Save src/hello_llm.py, run three questions, save the outputs. Confirm .env is gitignored.

~45 min

3

Write ADR v1

Fill the Solution Framing Canvas into docs/adr/0001-capstone-framing.md — all seven fields.

~60 min

Key Takeaways

What to carry into Week 2

Think in systems

Optimise the whole loop — not just model accuracy.

Match the tool

RAG, fine-tune, long-context, or agent — pick by constraint.

Measure honestly

Eight KPIs, and a clear line on when NOT to use AI.

Decide & document

One living ADR, evolved across all 30 weeks.

A marathon, not a sprint — show up consistently. Pre-read for W2 (async Python) is in Slack.

Q & A

Thank you